

1. Overview & Instruction Set

A single-cycle MIPS processor executes every instruction in exactly one clock cycle. It typically implements a simplified subset of MIPS ISA, including:

- **R-type arithmetic:** `add`, `sub`, `and`, `or`, `slt`
 - **Load/store:** `lw`, `sw`
 - **Branch:** `beq` [\[courses.cs...ington.edu\]](https://courses.cs.toronto.edu/~jason/courses/283S17/lectures/04-mips.html), [\[cs.gmu.edu\]](https://www.cs.gmu.edu/~kriszsan/teaching/2017-fall/2017-fall-283-lectures/04-mips.html)
-

2. Datapath Structure

The datapath connects several core components:

- **Program Counter (PC):** fetches instructions from instruction memory.
 - **Instruction Memory:** 32-bit instruction fetch.
 - **Register File:** 32 registers with two read ports and one write port.
 - **Sign-Extend Unit:** Extends 16-bit immediate to 32-bit.
 - **ALU:** Performs arithmetic, logical, and branch operations.
 - **Data Memory:** For `lw` and `sw`.
 - **Multiplexers (Muxes):**
 - To select between `rt` and `rd` for destination register (`RegDst`).
 - To choose ALU operand: register or immediate (`ALUSrc`).
 - To select between ALU result or memory data for register write-back (`MemtoReg`).
 - **Shift-left-2 & Adder:** For branch target address calculation.
 - **PC Mux:** Chooses between `PC+4` or branch target based on `Branch` and `Zero`.
[\[cs.gmu.edu\]](https://www.cs.gmu.edu/~kriszsan/teaching/2017-fall/2017-fall-283-lectures/04-mips.html), [\[blog.jyoti...rakash.org\]](https://blog.jyotiirakash.org/2017/08/28/mips-datapath/)
-

3. Control Signals

The main control unit decodes the opcode to generate signals that guide the datapath:

- **RegDst:** 1 for R-type (writes to `rd`), 0 otherwise.
- **ALUSrc:** 1 if second ALU input is immediate (`I`-type), else register.
- **MemtoReg:** 1 for `lw`, else write ALU result.
- **RegWrite:** Enables writing to register file.
- **MemRead / MemWrite:** For data memory access.
- **Branch:** For `beq`; combined with ALU Zero for PC selection.
- **ALUOp:** Tells ALU to perform correct operation (e.g., add, subtract, and, or)
[\[blog.jyoti...rakash.org\]](https://blog.jyotiirakash.org/2017/08/28/mips-datapath/)

These signals steer MUXes, enable memory/register writes, and configure ALU behavior.

4. Instruction Flow Breakdown

Here's how each instruction class flows through the datapath:

R-type (e.g., `add rd, rs, rt`):

1. Fetch instruction; decode opcode/fields.
2. Read `rs`, `rt` from registers.
3. ALU performs operation defined by `funct` field.
4. Write result to `rd`.
5. $PC \leftarrow PC + 4$.
Control: `RegDst=1`, `ALUSrc=0`, `MemtoReg=0`, `RegWrite=1`, `ALUOp=R`
[\[cs.gmu.edu\]](http://cs.gmu.edu), [\[blog.jyoti...rakash.org\]](http://blog.jyoti...rakash.org)

Load Word (`lw rt, imm(rs)`):

- Read base address from `rs`, add sign-extended `imm` via ALU.
- Read memory at computed address.
- Write memory data into `rt`.
- $PC \leftarrow PC + 4$.
Control: `RegDst=0`, `ALUSrc=1`, `MemtoReg=1`, `RegWrite=1`, `MemRead=1`, `ALUOp=add` [\[blog.jyoti...rakash.org\]](http://blog.jyoti...rakash.org)

Store Word (`sw rt, imm(rs)`):

- Sign-extend `imm`, compute address with ALU.
- Store `rt` contents to data memory.
- $PC \leftarrow PC + 4$.
Control: `ALUSrc=1`, `MemWrite=1`, `ALUOp=add` [\[blog.jyoti...rakash.org\]](http://blog.jyoti...rakash.org)

Branch Equal (`beq rs, rt, imm`):

- Read `rs`, `rt`; ALU subtracts to check Zero.
 - If equal, $PC \leftarrow PC + 4 + (\text{sign-extended } imm \ll 2)$; else $PC \leftarrow PC + 4$.
Control: `ALUSrc=0`, `Branch=1`, `ALUOp=subtract` [\[blog.jyoti...rakash.org\]](http://blog.jyoti...rakash.org)
-

5. Formal Datapath Design Steps

As described in George Mason's "Designing a Single-Cycle Datapath":

1. Analyze ISA to determine required register transfers.
 2. Select components: ALU, register file, memory, MUXes, etc.
 3. Assemble datapath meeting those transfers.
 4. For each instruction, determine control signal settings.
 5. Build control logic to generate those signals. [\[cs.gmu.edu\]](http://cs.gmu.edu)
-

6. Practical Implementations

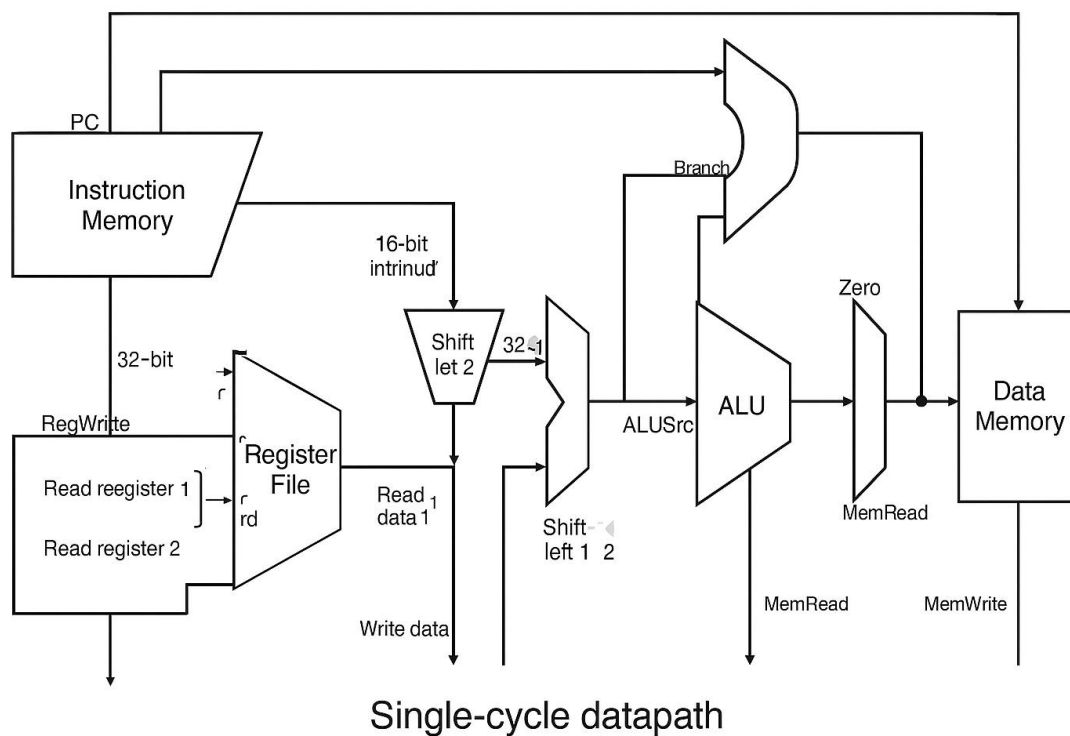
- **Logisim Tutorial** (Tufts) shows how to build and test datapath in simulator. [\[cs.tufts.edu\]](https://cs.tufts.edu)
 - **Verilog Example** by Hardware Lab NITC guides through modules for datapath, control, memory, register file. [\[hwlabnirc.github.io\]](https://hwlabnirc.github.io)
 - **GitHub Projects:**
 - Elzawawy's single-cycle MIPS in SystemVerilog. [\[github.com\]](https://github.com)
 - OmarMongy's repository with complete datapath and control in Verilog. [\[github.com\]](https://github.com)
-

7. Additional Resources & Visual Guides

- **University of Washington Slides:** Walk-through of single-cycle MIPS design principles. [\[courses.cs.washington.edu\]](https://courses.cs.washington.edu)
 - **Jyotiprakash Mishra's blog:** Detailed walkthrough of each control signal in the datapath. [\[blog.jyotirakash.org\]](https://blog.jyotirakash.org)
 - **YouTube Video by Nachum Danzig:** Visual explanation covering LW, ADD, BEQ in ~45 minutes. [\[youtube.com\]](https://youtube.com)
-

Quick Recap

- Single-cycle MIPS executes each instruction fully in one clock period.
- Datapath includes PC, instruction/data memory, register file, ALU, sign-extender, and MUXes.
- Control unit decodes opcodes to generate signals directing data flow and ALU behavior.
- Supports basic arithmetic, memory, and branch instructions.
- Ideal for learning before advancing to multi-cycle or pipelined designs.



Main Components

1. **Instruction Memory**
 - Fetches the 32-bit instruction using the Program Counter (PC).
2. **Register File**
 - Reads two source registers (r_s , r_t) and writes back results.
3. **Sign-Extend & Shift-left-2**
 - Extends 16-bit immediate to 32-bit and shifts for branch address.
4. **ALU**
 - Performs arithmetic, logic, and branch comparison.
5. **Data Memory**
 - Handles `lw` and `sw` instructions.
6. **Multiplexers (MUX)**
 - Control data flow for ALU input, destination register, and write-back.
7. **PC Update Logic**
 - Chooses between `PC+4` or branch target based on Branch & Zero signals.

Instruction Flow

- **Fetch:** PC → Instruction Memory → Instruction.
- **Decode:** Control unit sets signals; Register File reads operands.
- **Execute:**
 - ALU computes result or branch decision.
 - For `lw/sw`, ALU calculates memory address.
- **Memory Access:**
 - `lw`: Read from Data Memory.
 - `sw`: Write to Data Memory.
- **Write Back:**
 - Result (ALU or memory) written to Register File.
- **PC Update:**
 - Normally PC+4; if branch taken, $PC \leftarrow PC+4+(imm \ll 2)$.

✓ Control Signals Overview

Signal	Purpose
RegDst	Selects destination register (<code>rd</code> vs <code>rt</code>)
ALUSrc	Chooses ALU input: register or immediate
MemtoReg	Chooses data to write back: ALU or memory
RegWrite	Enables writing to register file
MemRead	Enables reading from data memory
MemWrite	Enables writing to data memory
Branch	Enables branch decision logic
ALUOp	Tells ALU which operation to perform

🔍 Instruction-wise Control Settings

1. R-type (e.g., `add rd, rs, rt`)

- **RegDst** = 1 (write to `rd`)
 - **ALUSrc** = 0 (second operand from register)
 - **MemtoReg** = 0 (write ALU result)
 - **RegWrite** = 1
 - **MemRead** = 0
 - **MemWrite** = 0
 - **Branch** = 0
 - **ALUOp** = 10 (function field decides operation)
-

2. Load Word (`lw rt, imm(rs)`)

- **RegDst** = 0 (write to `rt`)
- **ALUSrc** = 1 (second operand is immediate)

- **MemtoReg = 1** (write memory data)
 - **RegWrite = 1**
 - **MemRead = 1**
 - **MemWrite = 0**
 - **Branch = 0**
 - **ALUOp = 00** (perform addition for address)
-

3. Store Word (**sw** *rt*, *imm(rs)*)

- **RegDst = X** (don't care)
 - **ALUSrc = 1**
 - **MemtoReg = X**
 - **RegWrite = 0**
 - **MemRead = 0**
 - **MemWrite = 1**
 - **Branch = 0**
 - **ALUOp = 00** (addition for address)
-

4. Branch Equal (**beq** *rs*, *rt*, *imm*)

- **RegDst = X**
- **ALUSrc = 0**
- **MemtoReg = X**
- **RegWrite = 0**
- **MemRead = 0**
- **MemWrite = 0**
- **Branch = 1**
- **ALUOp = 01** (subtraction for comparison)

ALUOp Encoding

ALUOp	Meaning	Used For
00	Add	<i>lw</i> , <i>sw</i> (address calc)
01	Subtract	<i>beq</i> (comparison)
10	Use funct field	R-type instructions

Detailed Behavior

- **ALUOp = 00**
 - ALU performs **addition** regardless of funct field.
 - Needed for computing memory addresses in `lw` and `sw`.
 - Example: `lw $t0, 4($t1)` → ALU adds `$t1 + 4`.
- **ALUOp = 01**
 - ALU performs **subtraction**.
 - Used for branch comparison in `beq`.
 - Example: `beq $t0, $t1, label` → ALU subtracts `$t0 - $t1` and checks Zero flag.
- **ALUOp = 10**
 - ALU uses **funct field** from instruction to decide operation:
 - `funct = 100000` → `add`
 - `funct = 100010` → `sub`
 - `funct = 100100` → `and`
 - `funct = 100101` → `or`
 - `funct = 101010` → `slt`
 - Example: `add $s1, $s2, $s3` → ALU performs addition.